

Importing a Large Blinkit Dataset into MySQL Workbench

A step-by-step guide for loading ~38.5 lakh rows across five CSV files (~991 MB combined) using `LOAD DATA LOCAL INFILE`.

1. Why the GUI Import Wizard Does Not Work

MySQL Workbench includes a Table Data Import Wizard (right-click a table and choose Import). For small files (under ~100 MB), it works. For files larger than that, it becomes unreliable for the following reasons:

- Insertion is done row-by-row, so a 315 MB file can take several hours or never finish.
- The tool holds the entire file in memory, which often freezes Workbench.
- There is no reliable progress indicator, so failures happen silently.
- Numeric columns with empty values (such as `delivery_time_mins` for cancelled orders) trigger type errors that abort the import.

The reliable alternative is the SQL command `LOAD DATA LOCAL INFILE`, which streams the file directly into MySQL. A 315 MB file that hangs in the GUI loads in roughly 40 seconds using this command.

2. Prerequisites

- MySQL Server 8.0 or later, installed and running.
- MySQL Workbench 8.0 or later.
- Administrative access to your local MySQL instance.
- Five CSV files in a known folder (example path: `C:/Users/DELL/Downloads/dataset_blinkit/`).

Important: File paths in SQL must use forward slashes (/), never backslashes (\), even on Windows.

3. About the Dataset

The dataset contains five CSV files, one per year, all sharing the same 37-column schema.

File	Rows	Approx. Size
blinkit_orders_2022.csv	4,48,578	115 MB
blinkit_orders_2023.csv	6,00,000	157 MB
blinkit_orders_2024.csv	7,50,000	197 MB
blinkit_orders_2025.csv	13,87,456	367 MB
blinkit_orders_2026.csv	6,73,548	178 MB
Total	38,59,582	~991 MB

The one column to watch: `delivery_time_mins`

delivery_time_mins is an integer column, but it is empty for orders that were never delivered (Cancelled, Returned, Partially Delivered). MySQL's default behaviour is to reject empty strings for INT columns, which causes the import to fail silently. This guide handles the column using a MySQL user variable trick that converts empty strings to NULL. Details in Step 4.

4. Step 1 — Enable local_infile on the Server

MySQL blocks local file loading by default. Enable it once before continuing.

1.1 Open a new query tab in Workbench and run:

```
SHOW VARIABLES LIKE 'local_infile';
```

The result will most likely show OFF.

1.2 Enable the setting:

```
SET GLOBAL local_infile = 1;
```

1.3 Verify it is now ON:

```
SHOW VARIABLES LIKE 'local_infile';
```

The Value column should now read ON.

Note: This setting resets if the MySQL service restarts. For a permanent change, add `local_infile=1` under the `[mysqld]` section of `my.ini` or `my.cnf`.

5. Step 2 — Configure the Workbench Client

Enabling `local_infile` on the server side alone is not sufficient. The Workbench client must also be configured to allow local file loading. This is the step where most people get stuck.

2.1 Open connection settings

From the top menu, go to: Database → Manage Connections.

2.2 Select your connection

In the left panel, select Local instance MySQL80 (or the name of your connection).

2.3 Open the Advanced tab

Click the Advanced tab near the bottom of the right panel.

2.4 Add the client option

Locate the Others: text box. Do not delete anything already inside it. Add the following line at the end, on its own line:

```
OPT_LOCAL_INFILE=1
```

2.5 Close the dialog

Click Close at the bottom.

2.6 Restart MySQL Workbench

Important: You must close MySQL Workbench completely and reopen it. Closing just the tab or the dialog is not sufficient. The client-side setting only takes effect after a full restart.

After restart, reconnect to your database.

6. Step 3 — Create the Database and Table

3.1 Create the database (if not already present)

```
CREATE DATABASE IF NOT EXISTS blinkit_db;  
USE blinkit_db;
```

Double-click `blinkit_db` in the left panel so that it becomes bold. This sets it as your active schema.

3.2 Create the table

Notice that `delivery_time_mins` is declared as `INT NULL`. This is required so that non-delivered orders can hold `NULL` instead of an integer.

```
USE blinkit_db;  
  
DROP TABLE IF EXISTS blinkit_orders;  
  
CREATE TABLE blinkit_orders (  
    order_id                VARCHAR(20),  
    order_timestamp          DATETIME,  
    order_date               DATE,  
    order_year               INT,  
    order_month              INT,  
    order_month_name         VARCHAR(15),  
    order_day_of_week        VARCHAR(15),  
    order_hour               INT,  
    user_id                  VARCHAR(15),  
    user_tier                 VARCHAR(15),  
    store_id                  VARCHAR(10),  
    store_city                VARCHAR(30),  
    store_zone                VARCHAR(15),  
    customer_pincode          VARCHAR(10),  
    delivery_slot             VARCHAR(20),  
    platform                  VARCHAR(15),  
    payment_method            VARCHAR(25),  
    order_status              VARCHAR(25),  
    cancellation_return_reason VARCHAR(50),  
    num_items_ordered         INT,  
    total_qty                 INT,  
    item_names_sample         VARCHAR(255),  
    mrp_total                 DECIMAL(10,2),  
    product_discount          DECIMAL(10,2),  
    subtotal                  DECIMAL(10,2),  
    promo_code                VARCHAR(20),  
    promo_discount            DECIMAL(10,2),  
    delivery_fee              DECIMAL(6,2),  
    tip                       DECIMAL(6,2),  
    cashback_earned           DECIMAL(8,2),  
    grand_total               DECIMAL(10,2),  
    delivery_time_mins         INT NULL,  
    order_rating              INT,  
    is_first_order            INT,  
    is_reorder                INT,  
    is_express_eligible       INT,  
    gstn_applied              INT  
);
```

Expected output: 0 row(s) affected. Refresh the Schemas panel and confirm that the `blinkit_orders` table now appears under Tables.

7. Step 4 — Load the CSV Files

Two details that determine success or failure

(a) The user variable trick for delivery_time_mins

In the LOAD DATA statement, the column delivery_time_mins is loaded into a user variable @delivery_time_mins (note the @ sign) instead of directly into the column. The SET clause at the end converts empty strings to NULL using NULLIF.

(b) Line endings

The CSV files use Unix line endings (\n), not Windows line endings (\r\n). Use LINES TERMINATED BY '\n'. Using the wrong terminator causes MySQL to treat the entire file as a single line, and the load reports Records: 0 with no errors.

Important: Load one file at a time. Wait for each LOAD DATA statement to complete before running the next one. Do not paste all five blocks at once.

Load 2022 (~4.48 lakh rows, ~10 seconds)

```
LOAD DATA LOCAL INFILE 'C:/Users/DELL/Downloads/dataset_blinkit/blinkit_orders_2022.csv'
INTO TABLE blinkit_orders
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS
(order_id, order_timestamp, order_date, order_year, order_month, order_month_name,
 order_day_of_week, order_hour, user_id, user_tier, store_id, store_city, store_zone,
 customer_pincode, delivery_slot, platform, payment_method, order_status,
 cancellation_return_reason, num_items_ordered, total_qty, item_names_sample,
 mrp_total, product_discount, subtotal, promo_code, promo_discount, delivery_fee,
 tip, cashback_earned, grand_total, @delivery_time_mins, order_rating,
 is_first_order, is_reorder, is_express_eligible, gstn_applied)
SET delivery_time_mins = NULLIF(@delivery_time_mins, '');
```

Expected: 448578 row(s) affected. Records: 448578. Warnings: 0.

Load 2023 (~6 lakh rows, ~15 seconds)

Use the same command as above but change the filename to blinkit_orders_2023.csv.

Expected: 600000 row(s) affected.

Load 2024 (~7.5 lakh rows, ~20 seconds)

Same command, filename blinkit_orders_2024.csv.

Expected: 750000 row(s) affected.

Load 2025 (~13.87 lakh rows, ~40 seconds)

This is the largest file. The status bar may show Executing... for close to a minute, which is normal. Filename: blinkit_orders_2025.csv.

Expected: 1387456 row(s) affected.

Load 2026 (~6.73 lakh rows, ~22 seconds)

Same command, filename blinkit_orders_2026.csv.

Expected: 673548 row(s) affected.

8. Step 5 — Verify the Import

5.1 Total row count

```
SELECT COUNT(*) AS total_rows FROM blinkit_orders;
```

Expected: 3,859,582.

5.2 Rows per year

```
SELECT
  order_year,
  COUNT(*) AS total_orders,
  FORMAT(SUM(grand_total), 2) AS total_gmv_inr,
  FORMAT(AVG(grand_total), 2) AS avg_order_value
FROM blinkit_orders
GROUP BY order_year
ORDER BY order_year;
```

order_year	total_orders	approx. GMV	avg order value
2022	4,48,578	~33 Cr	~739
2023	6,00,000	~48 Cr	~811
2024	7,50,000	~68 Cr	~917
2025	13,87,456	~157 Cr	~1,132
2026	6,73,548	~84 Cr	~1,256

5.3 Confirm NULL handling for delivery_time_mins

```
SELECT
  order_status,
  COUNT(*) AS orders,
  COUNT(delivery_time_mins) AS with_delivery_time,
  COUNT(*) - COUNT(delivery_time_mins) AS null_delivery_time
FROM blinkit_orders
GROUP BY order_status;
```

Only Delivered orders should have delivery_time_mins populated. Cancelled, Returned, and Partially Delivered orders should show NULL delivery times.

9. Common Errors and Fixes

Error: The used command is not allowed with this MySQL version

Cause: OPT_LOCAL_INFILE=1 was not saved in the Workbench connection settings, or Workbench was not restarted after the change.

Fix: Go to Database → Manage Connections → Advanced. Confirm the line is present in the Others box. Close Workbench completely and reopen it.

Error: The load succeeds but Records: 0 is reported

Cause: Line endings do not match the file.

Fix: Change `LINES TERMINATED BY '\r\n'` to `LINES TERMINATED BY '\n'`, or vice versa. To check the line ending: open the CSV in Notepad++ and use View → Show Symbol → Show End of Line. CRLF means `\r\n`. LF means `\n`.

Error: Incorrect integer value: " for column 'delivery_time_mins'

Cause: The LOAD DATA statement is writing directly to `delivery_time_mins` without the user variable trick.

Fix: Use `@delivery_time_mins` in the column list and add `SET delivery_time_mins = NULLIF(@delivery_time_mins, '')` at the end of the LOAD DATA statement.

Error: File not found (Errcode 2 - No such file or directory)

Cause: Incorrect file path.

Fix: Use forward slashes in the path (`C:/Users/...`). Confirm the file exists and the name is spelled correctly.

Error: No database selected

Cause: Active schema is not set.

Fix: Run `USE blinkit_db;` at the top of the query, or double-click `blinkit_db` in the left panel so that it becomes bold.

Import hangs indefinitely or Workbench freezes

Cause: You are still using the GUI Import Wizard on a large file.

Fix: Cancel the import and use `LOAD DATA LOCAL INFILE` as described in this guide.

Error: Data too long for column

Cause: A VARCHAR column is smaller than the actual data length.

Fix: Widen the column, for example: `ALTER TABLE blinkit_orders MODIFY item_names_sample VARCHAR(500);`

10. Post-Import Recommendations

10.1 Add indexes for query performance

Without indexes, every query scans all 38.5 lakh rows. The following six indexes cover the most common analytical queries and reduce query times significantly.

```
CREATE INDEX idx_order_date ON blinkit_orders(order_date);
CREATE INDEX idx_order_year ON blinkit_orders(order_year);
CREATE INDEX idx_store_city ON blinkit_orders(store_city);
CREATE INDEX idx_user_tier ON blinkit_orders(user_tier);
CREATE INDEX idx_status ON blinkit_orders(order_status);
CREATE INDEX idx_payment ON blinkit_orders(payment_method);
```

Each index takes roughly 30 to 90 seconds to create.

10.2 Create summary views

Precomputed views make dashboards load faster because heavy aggregations are calculated once and reused.

```
CREATE VIEW v_city_summary AS
SELECT
    store_city,
    COUNT(*) AS total_orders,
    SUM(grand_total) AS gmvt,
    AVG(grand_total) AS avg_order_value,
    AVG(delivery_time_mins) AS avg_delivery_mins
FROM blinkit_orders
GROUP BY store_city;

CREATE VIEW v_monthly_gmv AS
SELECT
    order_year,
    order_month,
    order_month_name,
    COUNT(*) AS orders,
    SUM(grand_total) AS gmv
FROM blinkit_orders
GROUP BY order_year, order_month, order_month_name
ORDER BY order_year, order_month;
```

10.3 Back up the database

Run the following command from a system terminal (not Workbench) to back up the entire database to a SQL file:

```
mysqldump -u root -p blinkit_db > blinkit_db_backup.sql
```

11. Summary of Key Points

- Do not use the GUI Import Wizard for files larger than 200 MB.
- local_infile must be enabled on both the server (SET GLOBAL local_infile = 1) and the Workbench client (OPT_LOCAL_INFILE=1).
- Fully close and reopen Workbench after changing the Advanced settings.
- Always use forward slashes in file paths, even on Windows.
- Handle empty numeric columns with the @variable and NULLIF pattern.
- Match the LINES TERMINATED BY clause to the actual line endings in the file (\n for Unix-style, \r\n for Windows-style).
- Load one file at a time and confirm the row count after each load.
- Add indexes on high-cardinality columns after all data is loaded, not before.